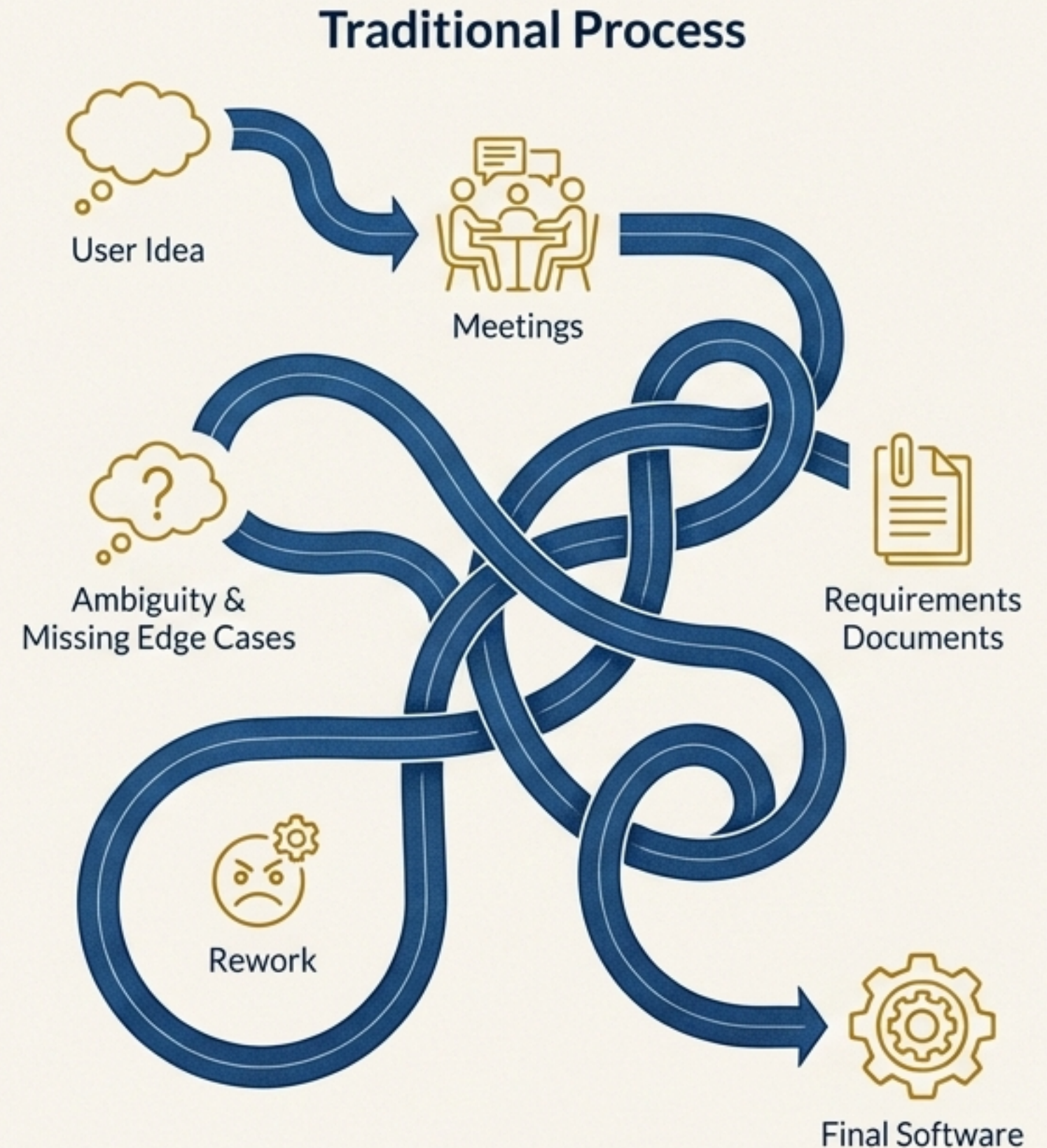


The Hardest Part of Software Isn't Code. It's Agreement.

We spend weeks, even months, in meetings trying to translate user desires into abstract requirements documents. Yet, ambiguity, missed edge cases, and shifting priorities often lead to costly rework.

The core challenge is a translation problem: users don't speak in formal requirements, and developers don't build from vague desires.

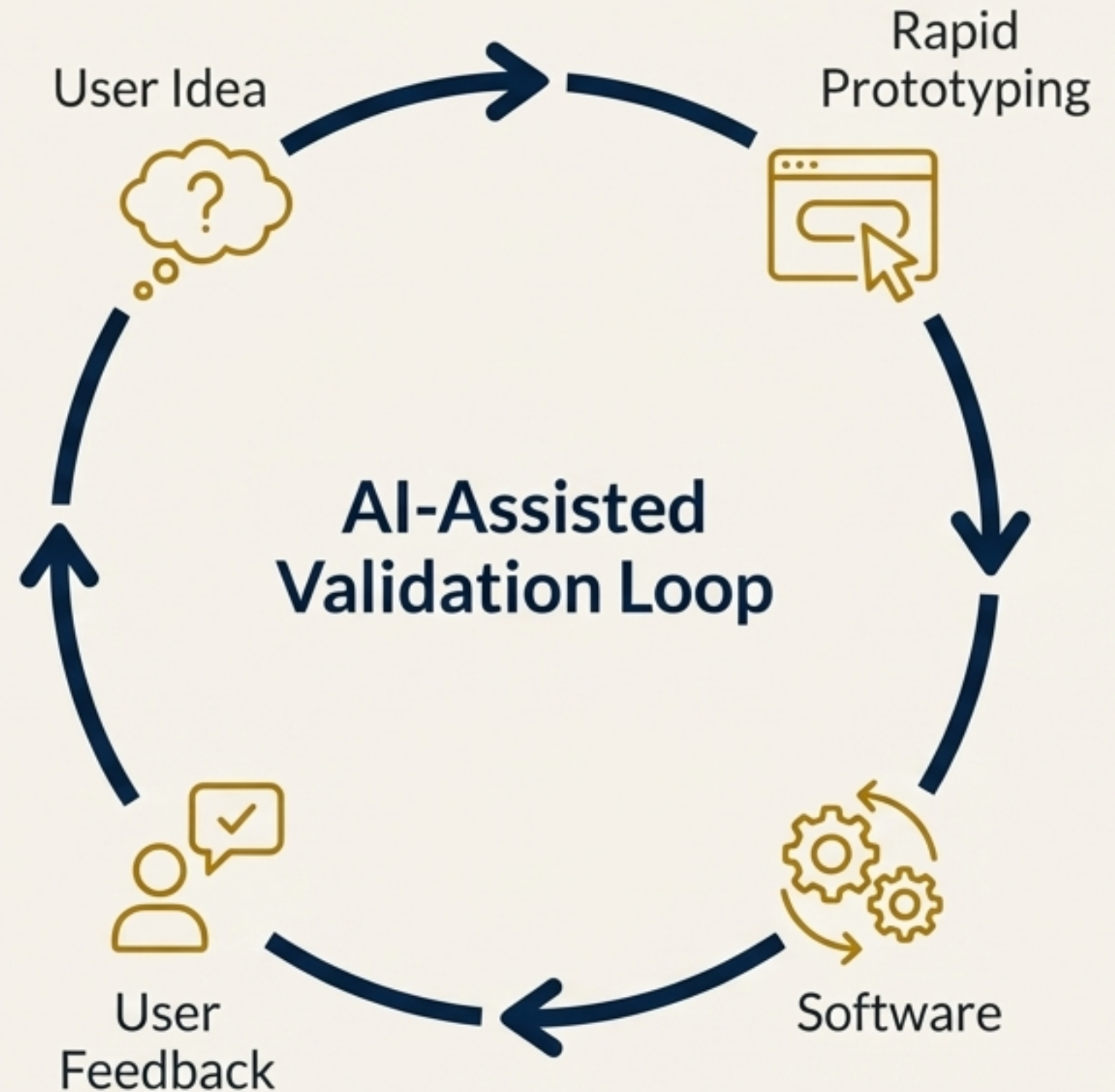
“Users often do not know what they want until they see something concrete.”



The Solution: A Shift in Philosophy from Gathering to Validating.

Instead of perfecting requirements in the abstract, we must accelerate the feedback loop. The fundamental principle is **speed to validation**.

By placing working prototypes in front of users early and often, we orient the project direction, validate assumptions, and engage stakeholders more deeply in a tangible process.



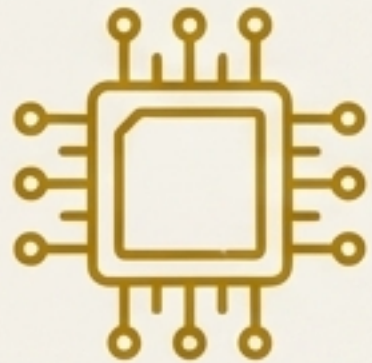
An AI-Assisted Framework in Three Critical Stages.

Our framework moves systematically from initial user desires to a fully validated, implementation-ready specification. This is not a list of steps, but a structured progression.



The First Step is Not Prototyping; It's Discovering Constraints.

Before any design begins, the analyst must establish the knowable outer perimeter of the project to avoid prototyping solutions that cannot exist within actual constraints. This documented perimeter prevents scope drift into infeasible territory.



Technology Constraints

Permissible platforms, integration patterns, authentication mechanisms.



Architecture & Data Sources

Current architecture, data availability, data quality, hosting constraints.



Legal, Security & Policy

GDPR/HIPAA, data handling policies, privacy classifications, retention requirements.

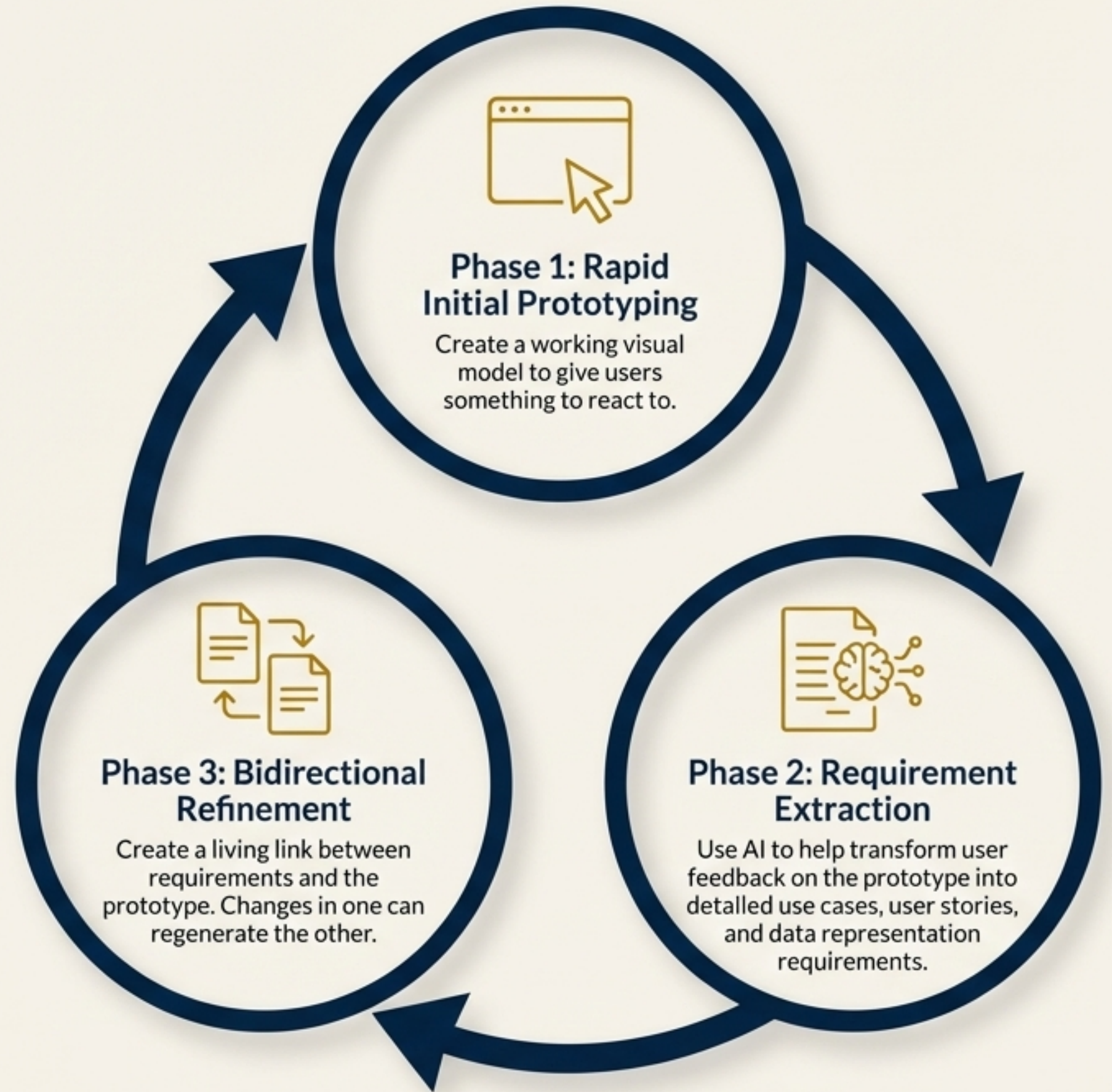


Operational Context

Physical environment, availability requirements, performance expectations.

Stage 1: We Transform User Desires into a Stable Prototype with the Discovery Loop

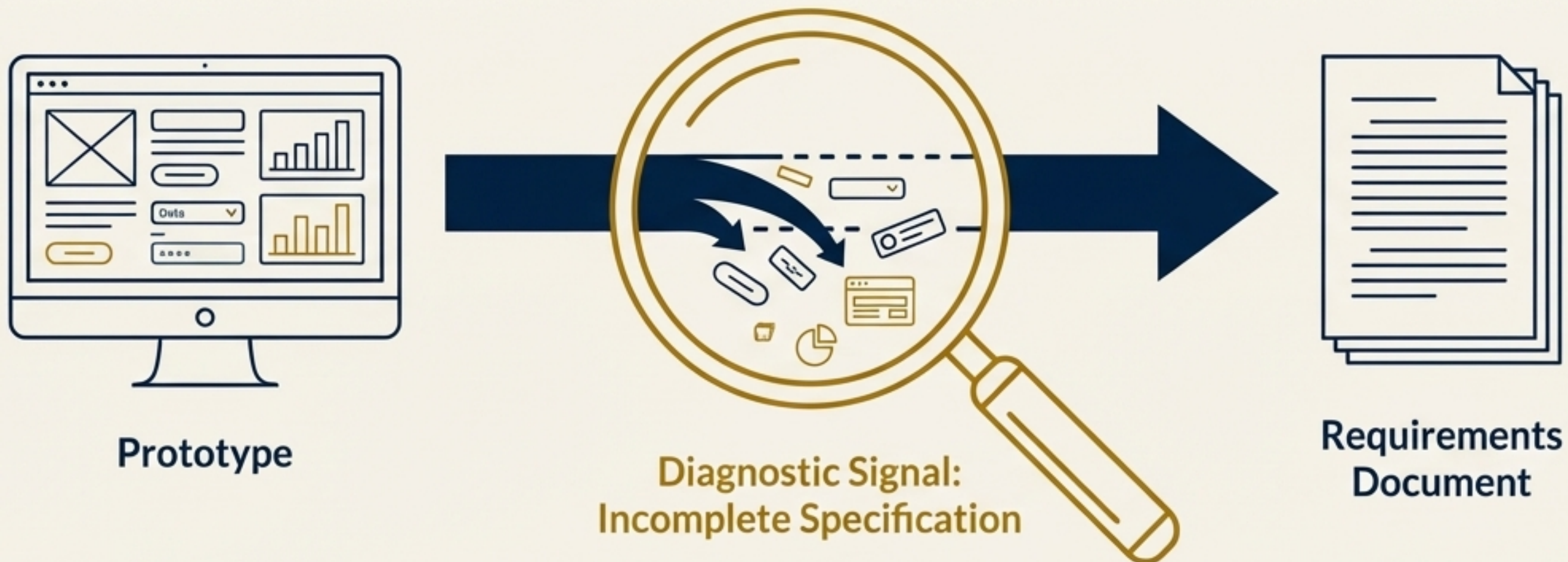
We begin by transforming initial user interviews into a functional visual prototype immediately. This tangible artifact becomes the center of a powerful feedback loop.



A Critical Insight: Information Loss is a Diagnostic Signal that Reveals Gaps

During the bidirectional refinement process, some information loss is inevitable when encoding the UI specification from the requirements document. This is not a bug; **it is a feature**.

These losses are diagnostic, highlighting exactly where requirements are incomplete or have not been adequately specified. They pinpoint what needs further refinement with the user, ensuring complete coverage.



Stage 2: We Achieve Certainty by Injecting Data and Generating AI-Driven Edge Cases.

With a stable UX, we move from abstract to concrete by injecting realistic data. This enables meaningful scenario testing and reveals unexpected complexities.

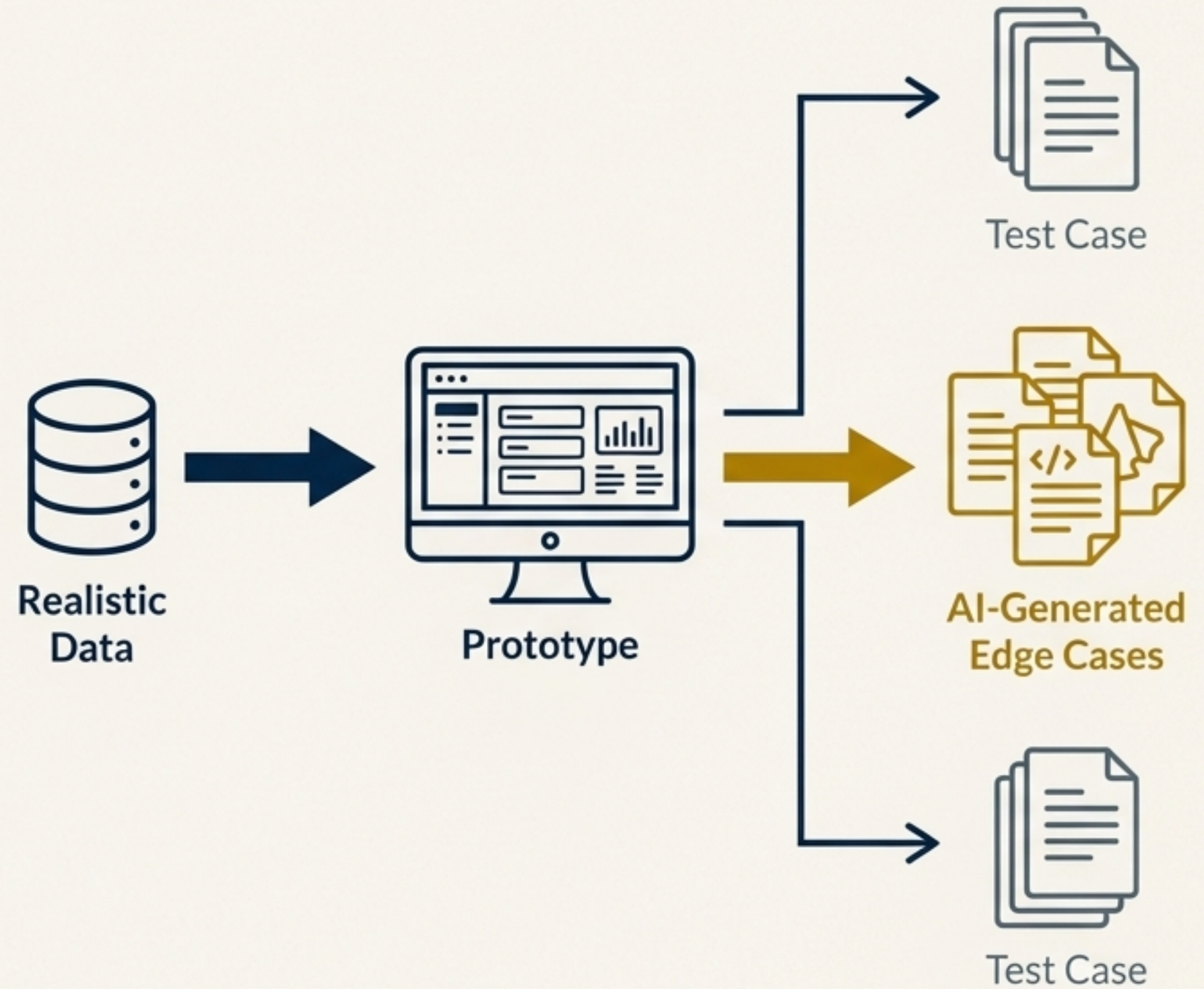
Key Activities

- **Phase 4: Data Injection**

Populate the prototype with realistic data to walk users through tangible scenarios.

- **Phase 5: AI-Assisted Edge Case Generation**

Here, AI's lack of domain expertise becomes an asset. It proposes scenarios that domain experts would never consider, revealing hidden gaps in requirements. While many will be invalid, some will uncover genuine blind spots.



We Lock In Formal Agreement Using Plain-Language Gherkin Scenarios.

Once the prototype, specifications, and edge cases are established, we formalize them into Gherkin statements. This plain-language, structured format is readable by both business and technical stakeholders, eliminating ambiguity.

The validation session becomes a rapid, binary process. Business users simply confirm if each scenario is correct, incorrect, or missing.

This achieves formal sign-off on the UX, interactions, and data requirements with maximum efficiency.

Given

The initial context (e.g., A user is logged in and on the dashboard).

When

The action or event (e.g., The user clicks the 'Export Report' button).

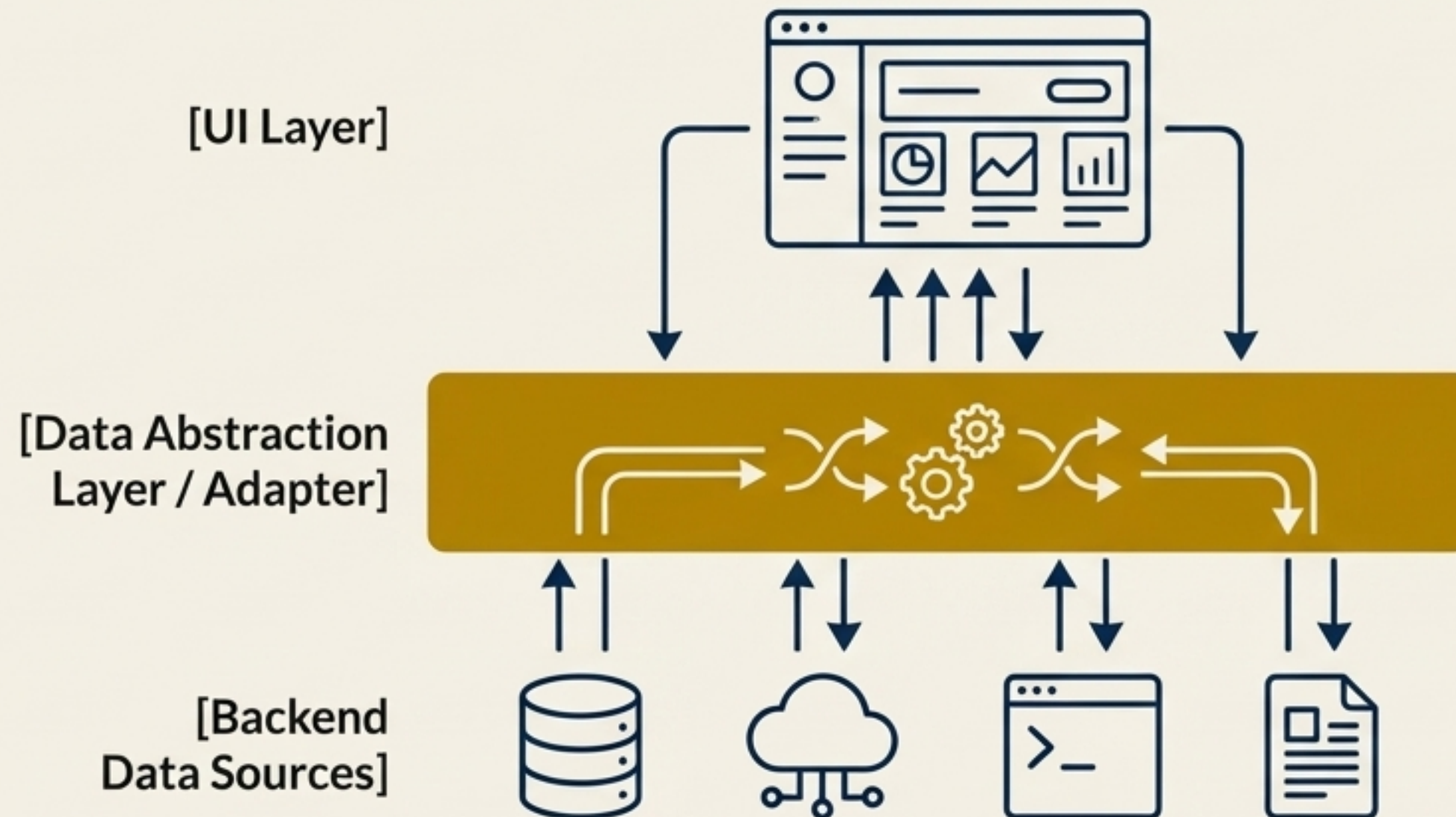
Then

The expected outcome (e.g., A CSV file named 'report.csv' is downloaded).



Stage 3: We Engineer for Flexibility with a Data Abstraction Layer.

To connect the validated UX to backend systems, we define a data model for the UI. The critical insight is that this UX data model does not need to map directly to the underlying storage data model. This separation of concerns (an Anti-Corruption Layer or Adapter Pattern) creates immense flexibility.



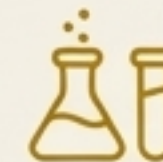
Benefits of Abstraction



UX can evolve independently of backend changes.



Multiple data sources can feed a single interface.



Testing is simplified with mock data contracts.



Backend technology can be migrated without impacting the UX.

The Final Blueprint is Validated and Ready for Two Paths to Implementation

With a complete, validated, and architecturally-sound specification, two primary implementation paths are available.



Path 1: Traditional Development with Enhanced Requirements

The specification is taken to a development team. Ambiguity is virtually eliminated, making implementation dramatically simpler and faster.

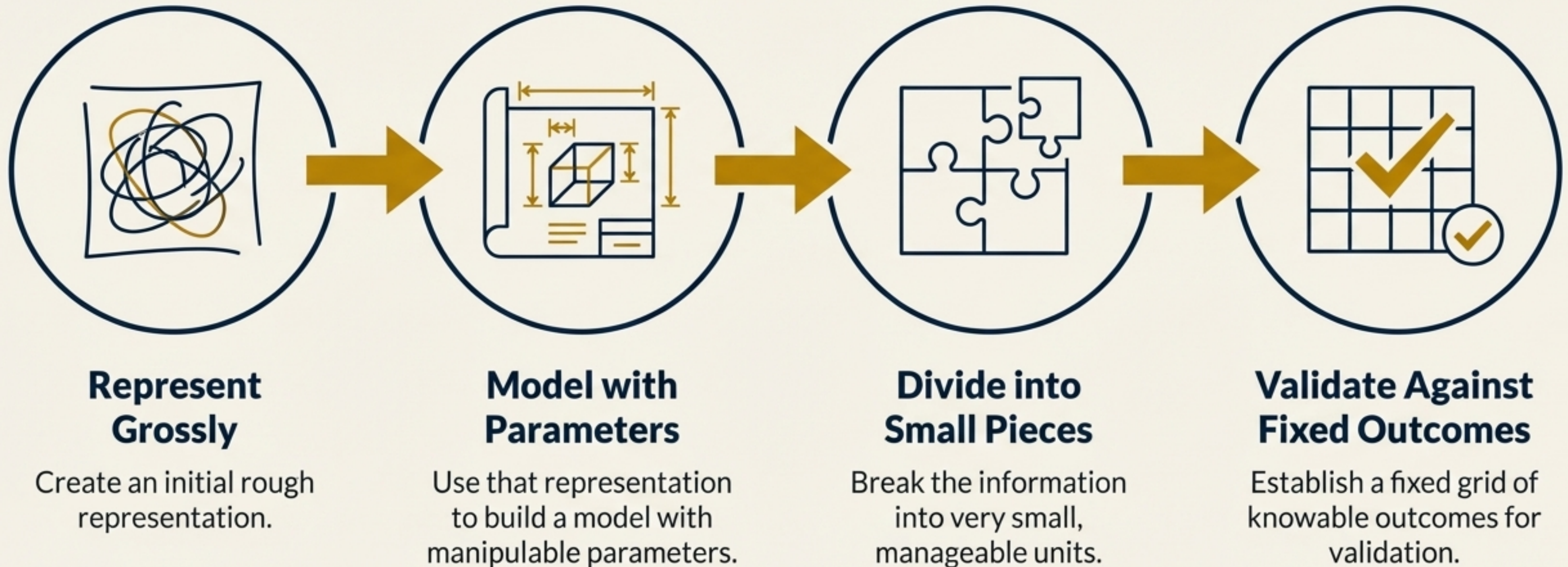


Path 2: AI-Assisted Full Implementation

Use AI to implement the production solution from the specification. The analyst must address production concerns intentionally omitted from the prototype (e.g., security, authentication).

The Entire Framework is Driven by a Universal Pattern for Solving Complex Problems.

Whether gathering requirements, creating tests, or generating code, the entire process follows a single, recursive pattern. This progression from a coarse understanding to fine-grained validation is the engine of the framework.



Key Insights from the Framework



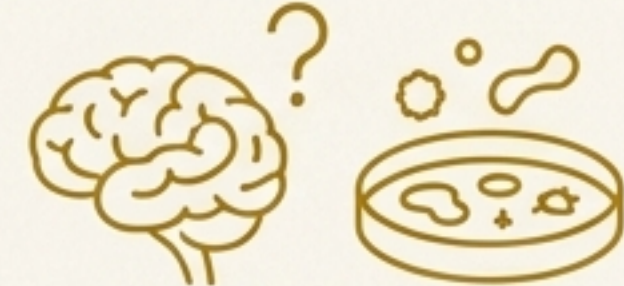
Users provide desires, not requirements

Prototypes accelerate the translation of feelings and pain points into formal specs.



Information loss reveals gaps

Failures to regenerate the prototype from requirements are diagnostic tools, not errors.



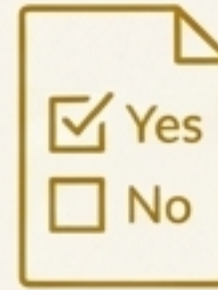
AI ignorance is an asset

AI's lack of domain expertise is a feature that uncovers edge cases insiders would miss.



Separation of concerns enables flexibility

Decoupling the UX data model from storage allows independent evolution of the front and back ends.



Gherkin enables efficient validation

Binary yes/no sessions with business users are fast and definitive.



The principle is universal

The pattern of Gross -> Model -> Divide -> Validate applies to requirements, tests, and code.

The Complete Framework at a Glance.

This table summarizes the primary activity and key output for each phase of the AI-assisted validation framework.

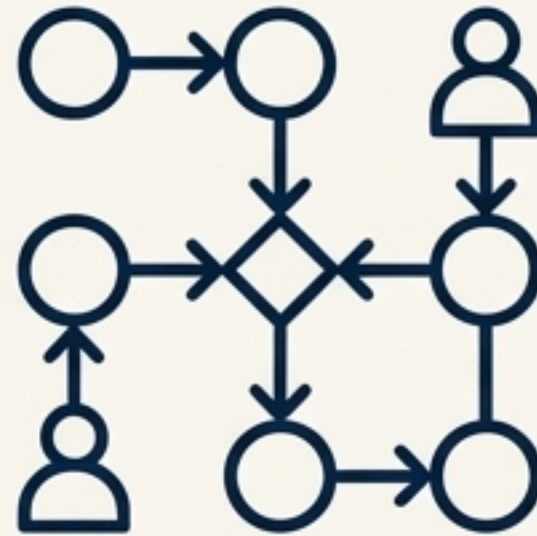
Phase	Primary Activity	Key Output
Phase 0: Constraints	Establish knowable project perimeter	Documented boundaries and constraints
Phase 1: Rapid Prototype	Quick visual prototype from initial input	Working prototype for validation
Phase 2: Extraction	Derive formal requirements from prototype	Use cases, user stories, data specs
Phase 3: Refinement	Bidirectional sync of requirements and UI	Gap identification, refined specs
Phase 4: Data Injection	Populate with realistic data and scenarios	Realistic prototype, initial test cases
Phase 5: Edge Cases	AI-generated boundary condition testing	Comprehensive test suite
Phase 6: Gherkin Tests	Formalize tests as Given-When-Then	Business-validated acceptance criteria
Phase 7: Data Abstraction	Define UX data model separate from storage	Data contracts, adapter patterns
Phase 8: Design System	Apply organizational visual standards	Production-ready UI mockups
Phase 9: Architecture	Confirm data source availability	Integration patterns, gap analysis
Phase 10: Implementation	Traditional dev or AI-assisted coding	Production system

Next Steps: From Concept to Practice

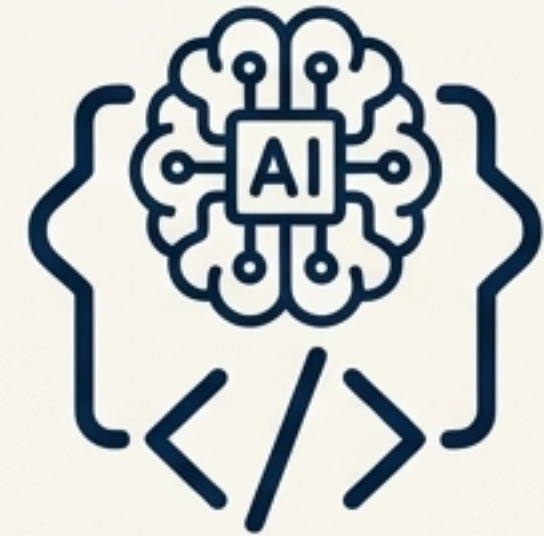
This framework provides a complete conceptual model for AI-assisted requirements and design. Further discussions can elaborate on specific areas based on your team's needs, including:



Recommended tooling and platforms for each phase.



Detailed workflows and team roles.



Advanced techniques for AI-assisted code generation.